compare(x: boolean, y: boolean)	int	S
valueOf(s: String)	Boolean	S
valueOf(b: boolean)	Boolean	S

compareTo(o: T)	int	

compare(d1: double, d2: double)	int	S
valueOf(s: String)	Double	S
valueOf(d: double)	Double	S

compare(x: int, y: int)	int	S
valueOf(s: String)	Integer	S
valueOf(i: int)	Integer	S

compare(x: long, y: long)	long	S
valueOf(s: String)	Long	S
valueOf(l: long)	Long	S

equals(object: Object)	boolean	
hashCode()	int	
toString()	String	

run()	void	

charAt(index: int)	char	
length()	int	
split(regex: String)	String[]	
toLowerCase()	String	
toUpperCase()	String	

currentTimeMillis()	long	S

print(obj: Object)	void	
printf(String format, Object... args)	PrintStream	
println()	void	
println(x: Object)	void	

sort(list: List<T>)	void	S
sort(list: List<T>, c: Comparator<T>)	void	S

compare(o1: T, o2: T)	int	
comparing(keyExtractor: Function<T,U>)	Comparator<T>	S

getKey()	K	
getValue()	V	
setValue(value: V)	V	

valueOf(arg0: String)	Enumeration	S
values()	Enumeration[]	S

add(e: E)	boolean	
add(index: int, element: E)	void	
contains(o: Object)	boolean	
forEach(action: Consumer<T>)	void	
get(index: int)	E	
of(elements: E...)	List<E>	S
remove(index: int)	E	
remove(o: Object)	boolean	
reversed()	List<T>	
size()	int	
sort(c: Comparator<T>)	void	

containsKey(key: Object)	boolean	
containsValue(value: Object)	boolean	
entrySet()	Set<Entry<K,V>>	
forEach(action: BiConsumer<K,V>)	void	
get(key: Object)	V	
keySet()	Set<K>	
put(key: K, value: V)	V	
putIfAbsent(key: K, value: V)	V	
values()	Collection<V>	

empty()	Optional<T>	S
get()	T	
ifPresent(action: Consumer<T>)	void	
ifPresentOrElse(action: Consumer<T>, emptyAction: Runnable)	void	
isPresent()	boolean	
of(t: T)	Optional<T>	S
ofNullable(t: T)	Optional<T>	S
orElse(other: T)	T	

empty()	OptionalDouble	S
getAsDouble()	double	
ifPresent(action: DoubleConsumer)	void	
ifPresentOrElse(action: DoubleConsumer, emptyAction: Runnable)	void	
isPresent()	boolean	
of(value: double)	OptionalDouble	S
orElse(other: double)	double	

nextInt(origin: int, bound: int)	int	

accept(t: T, u: U)	void	

accept(t: T)	void	

accept(value: double)	void	

apply(t: T)	R	

test(t: T)	boolean	

applyAsDouble(value: T)	double	

applyAsInt(value: T)	int	

getDayOfMonth()	int	
getDayOfYear()	int	
getMonth()	Month	
getMonthValue()	int	
getYear()	int	
now()	LocalDate	S
of(year: int, month: int, dayOfMonth: int)	LocalDate	S
parse(text: CharSequence)	LocalDate	S

getHour()	int	
getMinute()	int	
getSecond()	int	
now()	LocalTime	S
of(hour: int, minute: int, second: int)	LocalTime	S
parse(text: CharSequence)	LocalTime	S

averagingDouble(mapper: ToDoubleFunction<T>)	Collector<T,?,Double>	S
counting()	Collector<T,?,Long>	S
groupingBy(classifier: Function<T,K>)	Collector<T,?,Map<K,List<T>>>	S
joining(delimiter: CharSequence)	Collector<CharSequence,?,String>	S
mapping(m: Function<T,U>, ds: Collector<U,A,R>)	Collector<T,?,R>	S
partitioningBy(predicate: Predicate<T>)	Collector<T,?,Map<Boolean,List<T>>>	S
summingDouble(mapper: ToDoubleFunction<? super T>)	Collector<T,?,Double>	S
toList()	Collector<T,?,List<T>>	S
toMap(keyM: Function<T,K>, valueM: Function<T,U>)	Collector<T,?,Map<K,U>>	S
toSet()	Collector<T,?,Set<T>>	S

average()	OptionalDouble	
sum()	double	

average()	OptionalDouble	
sum()	int	

allMatch(predicate: Predicate<T>)	boolean	
anyMatch(predicate: Predicate<T>)	boolean	
collect(collector: Collector<T,A,R>)	R	
count()	long	
distinct()	Stream<T>	
filter(predicate: Predicate<T>)	Stream<T>	
findAny()	Optional<T>	
findFirst()	Optional<T>	
flatMap(mapper: Function<T,Stream<R>>)	Stream<R>	
forEach(action: Consumer<T>)	void	
limit(maxSize: long)	Stream<T>	
map(mapper: Function<T,R>)	Stream<R>	
mapToDouble(mapper: ToDoubleFunction<T,R>)	DoubleStream	
mapToInt(mapper: ToIntFunction<T,R>)	IntStream	
max(comparator: Comparator<T>)	Optional<T>	
min(comparator: Comparator<T>)	Optional<T>	
noneMatch(predicate: Predicate<T>)	boolean	
skip(n: long)	Stream<T>	
sorted()	Stream<T>	
sorted(comparator: Comparator<T>)	Stream<T>	
toList()	List<T>	
